UNIVERSITY OF MSILA
COMPUTER SCIENCE DEPARTMENT

L2-INF/INF-D
Object-oriented programming I
Problem set 2: Classes and Encapsulation

18/10/2026
two weeks

Exercise 1

Consider the following C program, which implements a counter the procedural way: a `struct` and separate functions that work on it.

```c
#include <stdio.h>
typedef struct { int value; } Counter;
void counter_init(Counter *c)      { c->value = 0; }
void counter_inc(Counter *c)       { c->value++; }
int  counter_get(const Counter *c) { return c->value; }
int main(void) {
  Counter c1, c2;
  counter_init(&c1);
  counter_inc(&c1);
  counter_inc(&c2);   // (a)
  c1.value = -7;      // (b)
  printf("%d\n", counter_get(&c1));
  return 0;
}
```

1. Compile and run the program. What does it print? Does the compiler or the program complain about anything?
2. Lecture 3 lists three problems of the procedural implementation of a stack. Name them.
3. Which problem is illustrated by line (a)? By line (b)?
4. Where in this program do you see the third problem?

Exercise 2: Class design

Rewrite the counter as a C++ class `Counter`.

- The state is a single integer `value`, initially 0.
- The behavior: `increment()`, `decrement()`, `reset()` and `getValue()`.
- The *invariant* is `value` $\geq$ 0: calling `decrement()` when the value is 0 must do nothing.
- The attribute must not be accessible from outside the class.

1. Write the class `Counter`.
2. Test it with the following `main` (add `#include <iostream>`). The expected output is `2` then `0`. Why is the first call to `decrement()` without effect?

```c
int main() {
  Counter c;
  c.decrement();
  c.increment();
  c.increment();
  c.increment();
  c.decrement();
  std::cout << c.getValue() << std::endl;
  c.reset();
  std::cout << c.getValue() << std::endl;
}
```

3. Add `c.value = -7;` to `main()`. What happens? Which problem of Exercise 1 is solved?

Exercise 3: Access control

```c
#include <iostream>
class Account {
  double balance = 0;
public:
  void deposit(double amount) {
    balance += amount;
  }
};
struct Point { int x; int y; };
class Size { int w; int h; };
int main() {
  Account a;
  a.deposit(100);            // (1)
  a.balance = 500;           // (2)
  Point p;
  p.x = 3;                   // (3)
  Size s;
  s.w = 8;                   // (4)
  std::cout << a.balance;    // (5)
}
```

1. Which lines of `main()` produce a compilation error? Explain each answer.

Exercise 4

A student writes this class and claims: "My attribute is private, so my class is encapsulated."

```
class BankAccount {
private:
  double balance = 0;
public:
  void setBalance(double b) {
    balance = b;
  }
  double getBalance() { return balance; }
};
```

1. What *invariant* should a bank account protect?
2. Show a short `main()` that breaks this invariant *without* touching `balance` directly.
3. Redesign the class: remove `setBalance()` and provide `deposit(double)` and `withdraw(double)`. Both return a `bool` telling whether the operation was accepted. A deposit must be positive, and a withdrawal must be positive and must not exceed the balance.
4. Run the following test. The expected output is `1`, `1`, `70`, `0`, `0`, `70` (one per line).

```
int main() {
  BankAccount a;
  std::cout << a.deposit(100) << std::endl;
  std::cout << a.withdraw(30) << std::endl;
  std::cout << a.getBalance() << std::endl;
  std::cout << a.withdraw(500) << std::endl;
  std::cout << a.deposit(-5) << std::endl;
  std::cout << a.getBalance() << std::endl;
}
```

Exercise 5: Finding invariants

1. For each class, write the invariant that every valid object must satisfy.
   - `Rectangle`: `width`, `height`;
   - `Time`: `hours`, `minutes`;
   - `Temperature` (in Kelvin): `kelvin`;
   - `Grade` (a mark out of 20): `value`;
   - `Queue` (circular array): `mem[CAPACITY]`, `head`, `count`.

Exercise 6: A queue

A *queue* stores its elements in FIFO order (First In, First Out). We implement it with a circular array. Write a class `Queue` with:

- private attributes: `int mem[CAPACITY]`, `int head` (index of the first element) and `int count` (number of elements). Use `#define CAPACITY 5`;
- methods: `isEmpty()`, `isFull()`, `size()`, `void enqueue(int)` and `int dequeue()`;
- `enqueue()` on a full queue and `dequeue()` on an empty queue do nothing (`dequeue()` returns 0 in that case, as in `Stack`);
- the element after the last one is stored at index `(head + count) % CAPACITY`.

1. Write the invariants of `Queue`.
2. Write the class `Queue`.
3. Test it with the following `main()`. The expected output is `5`, `1 2`, `5`, `3 4 5 7 8` (one per line).

```
int main() {
  Queue q;
  for (int i = 1; i <= 6; i++)
    q.enqueue(i);  // the 6th call is ignored
  std::cout << q.size() << std::endl;
  std::cout << q.dequeue() << " "
            << q.dequeue() << std::endl;
  q.enqueue(7);
  q.enqueue(8);
  std::cout << q.size() << std::endl;
  while (!q.isEmpty())
    std::cout << q.dequeue() << " ";
  std::cout << std::endl;
}
```

4. Why would it be dangerous to make `head` and `count` public?

Exercise 7: Design discussion

In Lecture 3, `Stack::pop()` returns `0` when the stack is empty, and `push()` silently does nothing when the stack is full.

1. Why is returning `0` from `pop()` a bad design?
2. Propose *two different* ways to redesign `pop()` so that the caller can tell whether it succeeded. Write the signature of each (the code of the body is not required).
3. Does your proposal respect encapsulation? Does it protect the invariant $0 \leq$ `top` $\leq$ `CAPACITY`?

